

Student Course Registration System – Project Guidelines

1. Project Objective

The objective of this project is to build a **Student Course Registration System** that manages students and courses.

The system allows students to **view courses, register for courses, drop courses, and maintain records in files.**

2. Functional Requirements

Course Management

- Add new courses

- View all available courses

- Update course seat availability

- Store course details in a file

Student Management

- Register students

- View student details

- Track enrolled courses

Enrollment Operations

- Enroll in a course

- Drop a course

- Prevent enrollment if seats are full

3. System Models

Course Model

Represents course details.

Attributes:

Course ID

Course Name

Instructor

Available Seats

Responsibilities:

Store course information

Track seat availability

Provide course details

Example

Course ID: C101

Course Name: Python Programming

Instructor: Dr. Kumar

Seats Available: 30

Student Model

Represents student details.

Attributes:

Student ID

Student Name

Email

Enrolled Courses

Responsibilities:

Store student information

Track courses registered by the student

Example

Student ID: S001

Name: Ravi

Enrolled Courses: C101

4. System Workflow

Step 1 – Start Application

Menu options:

- 1 View Courses
- 2 Add Course
- 3 Register Student
- 4 Enroll Course
- 5 Drop Course
- 6 Exit

Step 2 – View Courses

Display all available courses with seat availability.

Step 3 – Register Student

Collect student information and save it in a file.

Step 4 – Enroll Course

System checks:

- Course exists
- Student exists
- Seats available

Then updates seat count and student enrollment.

Step 5 – Drop Course

System removes the course from the student's enrollment list and increases available seats.

registration_services.py

Overview

The **registration_services** module contains the **core business logic** for the Student Course Registration System. It manages operations related to **student registration, course enrollment, course dropping, and retrieving course or student details**.

This module acts as the **service layer** between the application interface (menu-driven program) and the data storage (files containing course and student information).

Responsibilities of the Module

The `registration_services` module is responsible for:

- Managing student registration
- Retrieving available course information
- Handling course enrollment for students
- Processing course drop requests
- Updating course seat availability
- Validating student and course existence
- Handling system exceptions during operations

5. File Handling

Two files should be used.

Courses File

Fields:

- Course ID
- Course Name
- Instructor
- Available Seats

Students File

Fields:

- Student ID
- Name

Email

Enrolled Courses

6. Exception Handling

Handle the following errors:

Course Not Found

Student Not Found

Course Full

File Not Found

7. Project Structure

```
course_registration_system
├── main.py
├── models
│   ├── course.py
│   └── student.py
├── services
│   └── registration_service.py
├── utils
│   └── file_handler.py
├── exceptions
│   └── custom_exceptions.py
├── data
│   ├── courses.csv
│   └── students.csv
├── tests
│   └── test_registration.py
└── README.md
```

Test Case ID	Test Scenario	Precondition	Input Data	Expected Output
TC01	View available courses	Courses file contains course data	Select option: View Courses	System displays course details (Course ID, Name, Instructor, Seats Available).
TC02	Register a new student	System is running	Student ID, Name, Email	Student details are stored in the file; message "Student registered successfully" is displayed.
TC03	Enroll student in a course	Student exists; course has available seats	Valid Student ID and Course ID	Course added to student record; seat count decreases; message "Course enrollment successful" is displayed.
TC04	Enroll in a full course	Course seat availability = 0	Valid Student ID and Course ID	System displays "Enrollment failed: No seats available" .
TC05	Course not found	Student exists; invalid course ID	Student ID and Course ID (e.g., C999)	System displays "Error: Course not found" .
TC06	Student not found	Course exists; invalid student ID	Invalid Student ID and valid Course ID	System displays "Error: Student not found" .
TC07	Drop a registered course	Student is currently enrolled	Student ID and Course ID	Course removed from student record; seat count increases; message "Course dropped successfully" .
TC08	Course data file missing	Courses file does not exist	Select option: View Courses	System displays "Error: Course data file not found" .

8. Sample Input & Output

Sample Input

1. Courses Input Data

courses.csv

```

course_id,course_name,instructor,seats_available
C101,Python Programming,Dr.Kumar,30
C102,Data Science Fundamentals,Dr.Smith,25
C103,Machine Learning Basics,Dr.Andrew,20
C104,Web Development,Dr.Ravi,15
C105,Database Systems,Dr.John,10

```

Explanation

Field	Description
course_id	Unique ID for the course
course_name	Name of the course
instructor	Instructor teaching the course
seats_available	Remaining seats

2. Students Input Data

students.csv

```
student_id,name,email,enrolled_courses
S001,Ravi Kumar,ravi@gmail.com,
S002,Meena Priya,meena@gmail.com,
S003,Arjun Raj,arjun@gmail.com,
S004,Divya S,divya@gmail.com,
S005,Karthik R,karthik@gmail.com,
```

Explanation

Field	Description
student_id	Unique student ID
name	Student name
email	Contact email
enrolled_courses	Courses registered by the student

Program Start

```
===== Student Course Registration System =====
```

```
1 View Courses
2 Add Course
3 Register Student
4 Enroll Course
5 Drop Course
6 Exit
```

View Courses

Input

1

Output

Course ID : C101

Course Name : Python Programming

Instructor : Dr. Kumar

Seats Available : 30

Enroll Course

Input

Enter Student ID: S001

Enter Course ID: C101

Output

Course enrollment successful

Enroll in Full Course

Output

Error: No seats available for this course

Drop Course

Input

Enter Student ID: S001

Enter Course ID: C101

Output

Course dropped successfully